

The Ultimate Web Development Handbook



Introduction to JavaScript

What is JavaScript?

JavaScript is a **programming language** used to make web pages **interactive**. While **HTML** structures the page and **CSS** styles it, **JavaScript adds behavior**.

For example:

- Want to show a popup when a user clicks a button? Use JavaScript.
- Want to build a game, form validation, or fetch data from a server? Use JavaScript.

JavaScript runs **in the browser**, meaning it executes on the user's device.

Why JavaScript?

- Works on **all modern browsers**
 - Essential for **front-end development**
 - Used by frameworks like **React, Vue, Angular**
 - Can also be used on the **backend** (Node.js)
-

Where Can You Write JavaScript?

There are 3 common ways to write JavaScript in a webpage:

Inline (Not recommended)

```
<button onclick="alert('Hello!')">Click Me</button>
```

In a `<script>` tag inside HTML

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Hello World</h1>
    <script>
      console.log("JavaScript is working!");
    </script>
  </body>
</html>
```

External JavaScript File (Recommended)

```
<!-- index.html -->
<!DOCTYPE html>
<html>
  <body>
    <h1>Hello</h1>
    <script src="script.js"></script>
  </body>
</html>
```

```
// script.js
console.log("Hello from external JS file!");
```

Your First JavaScript Code

Let's start with the simplest code that prints something:

```
console.log("Hello JavaScript");
```

`console.log()` is used to print messages to the **browser console**.

How to Open the Console

- Right-click on the page > Inspect
 - Go to the **Console** tab
 - You'll see outputs from `console.log()` here
-

JavaScript is Case-Sensitive

```
let x = 5;  
let X = 10;  
console.log(x); // 5  
console.log(X); // 10
```

Comments in JavaScript

Use comments to explain your code:

```
// This is a single-line comment  
  
/*  
  This is a  
  multi-line comment  
*/
```

Summary

- JavaScript makes your website interactive.
- You can write it inside HTML or in separate `.js` files.
- Use `console.log()` to test your code.
- Practice by writing simple scripts and watching them work in the browser.

Variables and Data Types in JavaScript

What is a Variable?

A **variable** is a named container for storing data. In JavaScript, we can declare variables using:

```
let name = "Harry";  
const age = 25;  
var city = "Delhi";
```

let vs const vs var

Keyword	Reassignable?	Block Scoped?	Hoisted?
let	Yes	Yes	Yes
const	No	Yes	Yes
var	Yes	No	Yes (but undefined)

Use let when:

- You plan to **change the value** later.

```
let score = 0;  
score = 10;
```

Use const when:

- The value **should not change**.

```
const pi = 3.14159;  
// pi = 3.14; Error
```

Avoid `var`

- It behaves inconsistently due to hoisting and lack of block scoping.

JavaScript Data Types

JavaScript has two categories of data types:

1. Primitive (Value) Data Types

These are **immutable** and **stored directly in memory**.

Data Type	Example
string	"Hello World"
number	42 , 3.14 , -100
boolean	true , false
null	null
undefined	undefined
bigint	12345678901234567890n
symbol	Symbol("id")

```
let name = "Harry";      // string  
let age = 25;            // number  
let isCool = true;       // boolean  
let noValue = null;      // null  
let notDefined;          // undefined
```

Note: `typeof null` is `"object"` due to a long-standing bug in JS.

2. Non-Primitive (Reference) Data Types

These hold **references** to memory, not actual values.

Type	Example
Object	<code>{ name: "Harry" }</code>
Array	<code>[1, 2, 3]</code>
Function	<code>function() {}</code>
Date, RegExp, etc.	Built-in Objects

```
let person = { name: "Harry", age: 25 }; // Object
let colors = ["red", "blue", "green"]; // Array
let greet = function() { console.log("Hi") }; // Function
```

Differences Between Primitive and Reference Types

Feature	Primitive	Reference
Stored as	Value	Memory address (reference)
Mutable?	Immutable	Mutable
Copied as	Value	Reference

```
let a = 10;
let b = a; // Copy by value
b = 20;
console.log(a); // 10 (unchanged)

let obj1 = { x: 1 };
```

```
let obj2 = obj1; // Copy by reference
obj2.x = 2;
console.log(obj1.x); // 2 (both point to same object)
```

typeof Operator

Use `typeof` to check the type of a variable:

```
console.log(typeof "Hello"); // string
console.log(typeof 100); // number
console.log(typeof true); // boolean
console.log(typeof undefined); // undefined
console.log(typeof null); // object (quirk!)
console.log(typeof {}); // object
console.log(typeof []); // object (array is also object)
console.log(typeof function(){}); // function
```

Summary

- Use `let` for changeable values, `const` for constants.
- Understand the difference between **primitive** (copied by value) and **reference** types (copied by reference).
- `typeof` is useful but not perfect (e.g., `typeof null === "object"`).

Naming Variables in JavaScript

Rules for Naming Variables

1. Variable names must begin with a letter, underscore `_`, or dollar sign `$`.

Valid examples:

```
let name;  
let _count;  
let $price;
```

Invalid example:

```
let 1user; // Invalid: cannot start with a number
```

2. **Use camelCase for variable names.** In JavaScript, the convention is to use **camelCase**, where the first word is lowercase and each new word starts with an uppercase letter:

```
let userName;  
let totalAmount;  
let isLoggedIn;
```

3. **Be descriptive and meaningful.** Use names that describe the purpose of the variable:

```
let age = 25;           // Good  
let userAge = 25;       // Better  
let a = 25;             // Poor
```

4. **Avoid using reserved keywords.** JavaScript has reserved words that cannot be used as variable names:

```
let let;      // Invalid
let function; // Invalid
```

5. **No spaces or special characters.** Variable names must not contain spaces or symbols like `@`, `-`, or `#`:

```
let user name;    // Invalid
let user-name;    // Invalid
let user_name;    // Valid, but camelCase is preferred
```

Good Examples

```
let firstName = "John";
let totalPrice = 100;
let isAvailable = true;
```

Bad Examples

```
let x = "John";      // Not descriptive
let user_123_abc = 25; // Unclear naming
let 1user = "Alice";  // Invalid syntax
```

Boolean Variable Naming

When naming variables that store `true` or `false`, use prefixes like `is`, `has`, or `can` to make their intent clear:

```
let isLoggedIn = true;  
let hasAccess = false;  
let canEdit = true;
```

Summary

- Use camelCase.
- Choose meaningful names.
- Start names with a letter, `_`, or `$`.
- Avoid JavaScript keywords.
- Avoid spaces and special characters.